

Fase 10 — Replicación Física (Maestro-Esclavo)

Continuación directa de la Fase 9. `bdd-nodo01` y `bdd-nodo02` tienen MariaDB 10.11 instalado con el esquema `lab_bdd` idéntico y poblado en ambos nodos, y el snapshot `fase09-completa` está tomado. Esta fase convierte `bdd-nodo01` en el **MAESTRO** de replicación y `bdd-nodo02` en su **ESCLAVO**: cualquier escritura en el maestro se propagará automáticamente al esclavo en tiempo real usando binary log en formato ROW con GTID de MariaDB. Al finalizar, el laboratorio tendrá su primera capacidad de alta disponibilidad operativa: los datos del maestro quedan preservados —y accesibles en modo lectura— en el esclavo de forma continua.

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Explicar el papel del binary log en la replicación física y las diferencias entre los formatos STATEMENT, ROW y MIXED, justificando por qué se elige ROW.
2. Describir los dos hilos del esclavo (I/O thread y SQL thread) y la responsabilidad individual de cada uno en el proceso de replicación.
3. Distinguir la replicación basada en posición de archivo (file+pos) de la basada en GTID, y articular por qué GTID simplifica la administración del esclavo.
4. Configurar el binary log con GTID en el nodo maestro mediante un archivo de configuración dedicado, sin alterar `50-server.cnf` más allá del ajuste de `bind-address`.
5. Crear un usuario de replicación aplicando el principio de mínimo privilegio (`REPLICATION SLAVE ON *.*` restringido a la IP exacta del esclavo).
6. Establecer el enlace maestro-esclavo con `CHANGE MASTER TO ... MASTER_USE_GTID = slave_pos` y verificar que ambos hilos quedan en estado `Yes`.
7. Verificar experimentalmente la propagación en tiempo real de INSERT, UPDATE, DELETE y ALTER TABLE del maestro al esclavo.
8. Comprobar que `read_only = ON` bloquea escrituras directas de usuarios no privilegiados en el esclavo.
9. Interpretar los campos clave de `SHOW SLAVE STATUS` y usar `Seconds_Behind_Master` como indicador de lag de replicación.
10. Cerrar la fase con el snapshot `fase10-completa` en ambos nodos.

B. Conceptos teóricos necesarios

1. El binary log y su rol en la replicación.

El binary log (binlog) es un archivo de registro secuencial que MariaDB escribe en el maestro cada vez que se modifica el estado de los datos: DML (`INSERT` , `UPDATE` , `DELETE`) y DDL (`CREATE TABLE` , `ALTER TABLE` , `DROP DATABASE` , etc.). Las consultas de solo lectura (`SELECT`) no se registran. Los archivos se numeran progresivamente (`mariadb-bin.000001` , `.000002` , ...) y un archivo índice (`mariadb-bin.index`) los rastrea. El parámetro `expire_logs_days` controla cuántos días se conservan antes de purgarse.

El binlog tiene tres propósitos principales en un entorno de producción:

- **Replicación:** el esclavo lee sus eventos para reproducirlos localmente.
- **Recuperación punto-en-tiempo** (point-in-time recovery): restaurar una copia de seguridad y aplicar los eventos del binlog hasta un instante preciso.
- **Auditoría de cambios:** reconstrucción forense de qué cambió y cuándo.

2. Formatos del binary log.

Formato	Descripción	Ventajas	Inconvenientes
STATEMENT	Registra la sentencia SQL tal cual fue ejecutada.	Compacto, legible con <code>mysqlbinlog</code> .	Inseguro con funciones no deterministas (<code>NOW()</code> , <code>UUID()</code> , <code>RAND()</code>): el esclavo puede obtener valores diferentes al ejecutar la misma sentencia.
ROW	Registra los valores concretos (imagen antes y después) de cada fila afectada.	Completamente determinista; correcto para cualquier sentencia.	Mayor volumen en operaciones masivas (ej. <code>UPDATE ... WHERE 1</code> sin índice).
MIXED	Usa STATEMENT por defecto; cambia a ROW automáticamente ante sentencias no deterministas.	Balance entre los dos anteriores.	Menos predecible en tamaño del log; no recomendado para GTID.

Este laboratorio usa `ROW` exclusivamente, que es el estándar recomendado para producción, el único seguro con GTID y el requerido por las topologías multi-maestro de la Fase 11.

3. Los dos hilos del esclavo.

La replicación funciona con dos hilos independientes que corren permanentemente en el proceso MariaDB del esclavo:

- **Hilo I/O** (*IO thread*): actúa como cliente MariaDB. Se conecta al maestro usando el usuario de replicación, solicita los eventos del binlog a partir de la última posición GTID conocida, los descarga y los almacena en el **relay log** local del esclavo. Su estado se reporta en `SHOW SLAVE STATUS` como `Slave_IO_Running`.
- **Hilo SQL** (*SQL thread*): lee los eventos del relay log (ya locales, sin depender de la red) y los aplica al motor del esclavo, exactamente en el orden en que fueron generados en el maestro. Su estado se reporta como `Slave_SQL_Running`. La diferencia

entre el último evento descargado y el último aplicado determina el lag de replicación (`Seconds_Behind_Master`).

Esta arquitectura de dos hilos desacopla la descarga de la aplicación: si el hilo SQL falla por un error de datos, el hilo I/O puede seguir descargando eventos y el relay log acumula las transacciones pendientes para cuando se corrija el error.

4. GTID en MariaDB.

GTID (*Global Transaction Identifier*) es un identificador único que MariaDB asigna a cada transacción comprometida cuando el binary log está activo. En MariaDB (a diferencia de MySQL), el formato es:

Text

```
domain_id - server_id - sequence_number
```

Por ejemplo, `1-1-42` representa la transacción número 42 generada por el servidor con `server_id = 1` en el dominio de replicación `1`. Variables clave:

Variable	Nodo	Significado
<code>server_id</code>	Ambos	Identificador único del nodo (obligatorio; dos nodos con el mismo valor se rechazan).
<code>gtid_domain_id</code>	Ambos	Dominio de replicación; todos los nodos del laboratorio usan <code>1</code> .
<code>gtid_binlog_pos</code>	Maestro	Último GTID escrito en el binlog del maestro.
<code>gtid_slave_pos</code>	Esclavo	Último GTID aplicado por el esclavo (persistido en <code>mysql.gtid_slave_pos</code>).
<code>log_slave_updates</code>	Ambos	<code>ON</code> = el esclavo también escribe en su propio binlog los eventos recibidos del maestro; necesario para cadenas A→B→C y para failover.

Con `MASTER_USE_GTID = slave_pos`, el esclavo comunica al maestro su `gtid_slave_pos` y el maestro le envía exactamente los eventos que el esclavo todavía no ha aplicado, sin necesidad de conocer el nombre del archivo binlog ni la posición en bytes.

5. El relay log.

El relay log es el almacén local del esclavo para los eventos descargados del maestro. Usa el mismo formato binario que el binlog. Una vez que el hilo SQL aplica un evento, MariaDB puede purgarlo automáticamente (`relay_log_purge = ON`). El relay log también se divide en archivos numerados y tiene su propio archivo índice.

6. `read_only` en el esclavo.

El parámetro `read_only = ON` impide que usuarios sin el privilegio `SUPER` ejecuten DML o DDL directamente en el esclavo. Esto preserva la consistencia con el maestro: si un usuario externo modifica datos directamente en el esclavo, crea una divergencia que el binlog no conoce y que puede provocar errores en el hilo SQL cuando llegue la misma transacción del maestro (ej. `Duplicate entry` o `Row doesn't exist`). El hilo SQL del esclavo opera como `system user` interno, que está exento de `read_only` .

7. Asincronismo y su implicación en la consistencia.

La replicación clásica maestro-esclavo es **asíncrona**: el maestro confirma la transacción al cliente tan pronto como la escribe en su binlog local, sin esperar que ningún esclavo la haya descargado ni aplicado. Ventaja: menor latencia para el cliente. Riesgo: si el maestro cae justo después de confirmar y antes de que el esclavo descargue el evento, ese evento se pierde en el esclavo (*replication lag = data loss window*). La replicación semi-síncrona (plugin `rpl_semi_sync`) mitiga esto obligando al maestro a esperar el ACK de al menos un esclavo antes de confirmar; se introduce aquí como concepto pero su implementación corresponde a fases avanzadas.

C. Procedimiento paso a paso

Paso 1 — Iniciar ambas VMs y conectarse por SSH.

Paso 2 — Modificar `bind-address` en el maestro (nodo01).

El esclavo conecta al puerto 3306 del maestro a través de la red Host-Only. La configuración actual `bind-address = 127.0.0.1` impide conexiones desde otras IPs. Se cambia a `0.0.0.0` (todas las interfaces), lo cual es seguro en la red Host-Only aislada de este laboratorio.

Paso 3 — Crear el archivo de configuración de replicación del maestro.

Se crea `/etc/mysql/mariadb.conf.d/60-replication-master.cnf` con los parámetros de binary log, GTID y `server_id` . Al usar prefijo `60` , queda cargado después de `50-server.cnf` (cuya configuración base se conserva intacta) y antes de `99-lab-logs.cnf` .

Paso 4 — Reiniciar MariaDB en el maestro y verificar que el binlog quedó activo.

Paso 5 — Crear el usuario de replicación en el maestro.

Se crea `repl_user@'192.168.56.102'` con el privilegio mínimo `REPLICATION SLAVE`. Restringir el host a la IP exacta del esclavo es esencial: un usuario `repl_user@'%'` permitiría que cualquier máquina de la red se hiciera pasar por el esclavo.

Paso 6 — Registrar la posición GTID del maestro como referencia de partida.

Confirmar que `gtid_binlog_pos` inicial es vacío: ninguna transacción con GTID se ha aplicado desde que se habilitó el binlog. Este es el punto de sincronía: ambos nodos tienen los mismos datos históricos (cargados en la Fase 8) y el esclavo comenzará a recibir solo los cambios futuros del maestro.

Paso 7 — Crear el archivo de configuración de replicación del esclavo (nodo02).

Se crea `/etc/mysql/mariadb.conf.d/60-replication-slave.cnf` con `server_id = 2`, configuración del relay log y `read_only = ON`.

Paso 8 — Reiniciar MariaDB en el esclavo y verificar sus variables.

Paso 9 — Configurar el enlace de replicación en el esclavo con `CHANGE MASTER TO`.

Paso 10 — Iniciar los hilos de replicación con `START SLAVE`.

Paso 11 — Verificar el estado de replicación con `SHOW SLAVE STATUS`.

Confirmar que `Slave_IO_Running` y `Slave_SQL_Running` son `Yes` y `Seconds_Behind_Master` es `0`.

Paso 12 — Pruebas de propagación: INSERT, UPDATE, DELETE y ALTER TABLE en el maestro.

Ejecutar cada operación en nodo01 y confirmar en nodo02 que el cambio aparece en tiempo real.

Paso 13 — Prueba de `read_only`: intentar escribir directamente en el esclavo.

Confirmar que el esclavo rechaza escrituras de usuarios sin privilegio `SUPER`.

Paso 14 — Monitoreo de lag y posición GTID en ambos nodos.

Paso 15 — Apagar ambas VMs y tomar el snapshot `fase10-completa`.

D. Comandos completos

Los bloques **(host)** se ejecutan en PowerShell en Windows. Los bloques **(VM — nodoXX)** se ejecutan en una sesión SSH al nodo indicado. Los bloques SQL dentro de `sudo mariadb` se ejecutan directamente en el prompt del motor.

D.1 Iniciar ambas VMs y conectarse por SSH (host)

PowerShell

```
VBoxManage startvm "bdd-nodo01" --type headless
VBoxManage startvm "bdd-nodo02" --type headless
```

Esperar 30 segundos y abrir dos terminales SSH simultáneas:

PowerShell

```
# Terminal 1 - maestro
ssh bddadmin@192.168.56.101
# Terminal 2 - esclavo (segunda ventana de PowerShell)
ssh bddadmin@192.168.56.102
```

D.2 Modificar `bind-address` en el maestro (VM — nodo01)

El archivo `50-server.cnf` tiene `bind-address = 127.0.0.1`. Se cambia a `0.0.0.0` para que el esclavo pueda conectarse al puerto 3306 a través de la red Host-Only:

Bash

```
# Ver el valor actual antes de modificar
grep bind-address /etc/mysql/mariadb.conf.d/50-server.cnf
# Reemplazar la línea (el patrón cubre posibles espacios alrededor del =)
sudo sed -i \
    's/^bind-address[ ](:space:)*=.*\/bind-address          =
0.0.0.0/' \
    /etc/mysql/mariadb.conf.d/50-server.cnf
# Confirmar el cambio
grep bind-address /etc/mysql/mariadb.conf.d/50-server.cnf
```

Salida esperada después del cambio:

Text

```
bind-address          = 0.0.0.0
```

Si la línea `bind-address` no existe en `50-server.cnf` o está comentada, agregarla explícitamente con `echo 'bind-address = 0.0.0.0' | sudo tee -a /etc/mysql/mariadb.conf.d/50-server.cnf` y verificar con `grep bind-address /etc/mysql/*.cnf /etc/mysql/**/*.cnf` que no hay otro archivo que la defina.

D.3 Crear el archivo de configuración de replicación del maestro (VM — nodo01)

 Bash

```
sudo tee /etc/mysql/mariadb.conf.d/60-replication-master.cnf > /dev/null
<< 'EOF'

# =====
# Configuración de replicación — ROL: MAESTRO
# Nodo: bdd-nodo01 | IP: 192.168.56.101 | server_id: 1
# Fase 10 del Laboratorio BDD.
# Cargado después de 50-server.cnf (número de archivo mayor).
# =====

[mariadb]
# --- Identificación única del nodo (NUNCA debe repetirse en el
ecosistema) ---
server_id                = 1
# --- Binary log ---
# Prefijo de los archivos de binlog en /var/log/mysql/
log_bin                  = /var/log/mysql/mariadb-bin
# ROW: registra valores concretos de cada fila, no la sentencia SQL.
# Obligatorio para GTID y para replicación multi-maestro (Fase 11).
binlog_format            = ROW
# Retención automática de archivos binlog (en días)
expire_logs_days         = 7
# Tamaño máximo por archivo antes de rotar al siguiente
max_binlog_size          = 100M
# Incluir la sentencia SQL original en eventos ROW (facilita mysqlbinlog)
binlog_annotate_row_events = ON
# Garantiza que el binlog se escribe a disco en cada COMMIT
# (consistencia máxima; puede reducir rendimiento en disco lento)
sync_binlog              = 1
# --- GTID ---
# Dominio de replicación del laboratorio (todos los nodos usan 1)
gtid_domain_id           = 1
# El maestro también escribe en su binlog los eventos que recibe de otros
# (necesario para cadenas A→B→C y para failover controlado)
log_slave_updates        = ON
# --- Seguridad de red ---
# Evitar resolución DNS por hostname: más rápido y previene errores
# cuando el DNS no responde. Los GRANTS deben usar IPs, no nombres.
skip_name_resolve        = ON
EOF
```

Verificar que el archivo se creó correctamente:

 Bash

```
cat /etc/mysql/mariadb.conf.d/60-replication-master.cnf
```

D.4 Reiniciar MariaDB en el maestro y verificar el binary log activo (VM — nodo01)

 Bash

```
sudo systemctl restart mariadb  
sudo systemctl status mariadb --no-pager -l
```

La salida debe mostrar `active (running)` . Si el servicio no arranca, revisar el log:

 Bash

```
sudo journalctl -u mariadb --no-pager -n 40
```

Verificar que el binary log y las variables de replicación quedaron activos:

 Bash

```
sudo mariadb << 'EOF'  
-- ¿Está el binlog habilitado?  
SHOW VARIABLES LIKE 'log_bin';  
-- ¿Cuál es el formato?  
SHOW VARIABLES LIKE 'binlog_format';  
-- server_id, GTID y otras variables de replicación  
SHOW VARIABLES LIKE 'server_id';  
SHOW VARIABLES LIKE 'gtid_domain_id';  
SHOW VARIABLES LIKE 'log_slave_updates';  
SHOW VARIABLES LIKE 'sync_binlog';  
-- ¿El motor ahora escucha en 0.0.0.0?  
SHOW VARIABLES LIKE 'bind_address';  
-- Listar archivos binlog existentes (debe haber mariadb-bin.000001)  
SHOW BINARY LOGS;  
-- Estado completo del maestro: archivo actual, posición y GTID  
SHOW MASTER STATUS;  
EOF
```

Salida esperada de `SHOW MASTER STATUS` (los valores numéricos variarán):

Text

```
+-----+-----+-----+-----+-----+
| File           | Position | Binlog_Do_DB | Binlog_Ignore_DB |
Gtid_binlog_pos |
+-----+-----+-----+-----+-----+
| mariadb-bin.000001 |      351 |              |                  |
|                  |         |              |                  |
+-----+-----+-----+-----+-----+
```

El campo `Gtid_binlog_pos` estará vacío o mostrará `0-1-N` si el motor ya escribió algún evento interno. Anotar el valor exacto: es el punto de partida del esclavo.

D.5 Crear el usuario de replicación en el maestro (VM — nodo01)

Bash

```
sudo mariadb << 'EOF'

-- -----
-- Usuario de replicación
-- El host '192.168.56.102' restringe el acceso a la IP exacta
-- del esclavo (principio de mínimo privilegio).
-- Se usa mysql_native_password porque el protocolo de replicación
-- binaria no soporta auth_socket.
-- -----

CREATE USER IF NOT EXISTS 'repl_user'@'192.168.56.102'
  IDENTIFIED BY 'ReplUser_2025!';
GRANT REPLICATION SLAVE ON *.* TO 'repl_user'@'192.168.56.102';
FLUSH PRIVILEGES;
-- Verificar que el usuario fue creado con el host correcto
SELECT User, Host, plugin
FROM   mysql.user
WHERE  User = 'repl_user';
-- Verificar que tiene exactamente REPLICATION SLAVE y nada más
SHOW GRANTS FOR 'repl_user'@'192.168.56.102';
EOF
```

Salida esperada de `SHOW GRANTS` :

Text

```
GRANT REPLICATION SLAVE ON *.* TO 'repl_user'@'192.168.56.102' IDENTIFIED  
BY PASSWORD '...'
```

D.6 Registrar la posición GTID de partida (VM — nodo01)

Bash

```
sudo mariadb -e "SELECT @@global.gtid_binlog_pos AS gtid_inicio_maestro;"  
sudo mariadb -e "SHOW MASTER STATUS\G"
```

Punto de sincronía importante: en este momento `bdd-nodo01` tiene el binlog habilitado pero no ha recibido ninguna transacción de aplicación desde la habilitación. Ambos nodos tienen la misma base de datos `lab_bdd` con los datos de la Fase 8. El esclavo comenzará a replicar exactamente desde este punto en adelante.

Entre este paso y `START SLAVE` del Paso 10, no ejecutar ninguna operación DML ni DDL en `bdd-nodo01`. Si lo haces, esas transacciones quedarán en el binlog y el esclavo intentará aplicarlas sobre datos que ya tiene, generando un error `Duplicate entry` en el hilo SQL.

D.7 Crear el archivo de configuración de replicación del esclavo (VM — nodo02)

 Bash

```
sudo tee /etc/mysql/mariadb.conf.d/60-replication-slave.cnf > /dev/null
<< 'EOF'

# =====
# Configuración de replicación — ROL: ESCLAVO
# Nodo: bdd-nodo02 | IP: 192.168.56.102 | server_id: 2
# Fase 10 del Laboratorio BDD.
# =====

[mariadb]
# --- Identificación única del nodo ---
server_id                = 2
# --- Relay log: almacena eventos descargados del maestro ---
relay_log                = /var/log/mysql/mariadb-relay-bin
relay_log_index           = /var/log/mysql/mariadb-relay-bin.index
# Purgar automáticamente los segmentos del relay log ya aplicados
relay_log_purge          = ON
# --- Binary log del esclavo ---
# Necesario para log_slave_updates y para que este nodo pueda
# convertirse en maestro de otro esclavo en el futuro (failover)
log_bin                  = /var/log/mysql/mariadb-bin
binlog_format             = ROW
expire_logs_days          = 7
max_binlog_size           = 100M
binlog_annotate_row_events = ON
# Escribir en el binlog propio los eventos recibidos del maestro
log_slave_updates         = ON
# --- GTID (debe coincidir con el maestro) ---
gtid_domain_id           = 1
# --- Protección de escrituras directas ---
# Rechaza DML/DDL de usuarios sin privilegio SUPER
# El hilo SQL de replicación (system user) está exento
read_only                 = ON
# --- Seguridad de red ---
skip_name_resolve         = ON
EOF
```

Verificar el archivo:

 Bash

```
cat /etc/mysql/mariadb.conf.d/60-replication-slave.cnf
```

D.8 Reiniciar MariaDB en el esclavo y verificar sus variables (VM — nodo02)

 Bash

```
sudo systemctl restart mariadb
sudo systemctl status mariadb --no-pager -l
```

Verificar las variables clave del esclavo:

 Bash

```
sudo mariadb << 'EOF'
SHOW VARIABLES LIKE 'server_id';
SHOW VARIABLES LIKE 'read_only';
SHOW VARIABLES LIKE 'relay_log';
SHOW VARIABLES LIKE 'log_bin';
SHOW VARIABLES LIKE 'gtid_domain_id';
SHOW VARIABLES LIKE 'log_slave_updates';
-- Confirmar que aún no hay enlace configurado (resultado vacío)
SHOW SLAVE STATUS\G
EOF
```

`SHOW SLAVE STATUS` debe devolver un resultado vacío porque aún no se ejecutó `CHANGE MASTER TO`.

D.9 Configurar el enlace de replicación en el esclavo (VM — nodo02)

 Bash

```
sudo mariadb << 'EOF'

-- -----
-- CHANGE MASTER TO: define de dónde y cómo replicar.
--
-- MASTER_USE_GTID = slave_pos:
--   El esclavo enviará su gtid_slave_pos al maestro.
--   Como está vacío (ningún GTID aplicado aún), el maestro enviará
--   desde el inicio del binary log – exactamente lo que queremos,
--   porque los datos actuales en ambos nodos provienen de la Fase 8
--   (antes de que el binlog existiera) y solo se replicarán cambios
--   futuros.
-- -----

CHANGE MASTER TO
  MASTER_HOST      = '192.168.56.101',
  MASTER_PORT      = 3306,
  MASTER_USER      = 'repl_user',
  MASTER_PASSWORD  = 'ReplUser_2025!',
  MASTER_USE_GTID  = slave_pos;
-- Verificar que la configuración se almacenó (sin iniciar todavía)
SHOW SLAVE STATUS\G
EOF
```

En la salida de `SHOW SLAVE STATUS`, antes de iniciar, confirmar:

- `Master_Host: 192.168.56.101`
- `Master_User: repl_user`
- `Master_Port: 3306`
- `Using_Gtid: Slave_Pos`
- `Slave_IO_Running: No` (todavía no iniciado)
- `Slave_SQL_Running: No` (todavía no iniciado)

D.10 Iniciar los hilos de replicación (VM — nodo02)

 Bash

```
sudo mariadb -e "START SLAVE;"
# Dar 5 segundos para que los hilos establezcan la conexión inicial
sleep 5
sudo mariadb -e "SHOW SLAVE STATUS\G"
```

Campos críticos a verificar:

 Text

Slave_IO_Running: Yes	← hilo I/O conectado al maestro ✓
Slave_SQL_Running: Yes	← hilo SQL aplicando eventos ✓
Seconds_Behind_Master: 0	← sin retraso ✓
Master_Host: 192.168.56.101	
Master_User: repl_user	
Using_Gtid: Slave_Pos	
Gtid_IO_Pos:	← GTID hasta el que se descargó
Last_IO_Error:	← vacío = sin errores ✓
Last_SQL_Error:	← vacío = sin errores ✓

Si cualquiera de los dos hilos muestra **No**, revisar **Last_IO_Error** y **Last_SQL_Error** y consultar la sección F.

D.11 Pruebas de propagación en tiempo real (VM — nodo01 y nodo02)

Ejecutar cada bloque de nodo01 y verificar inmediatamente en nodo02.

Prueba 1 — INSERT

En **bdd-nodo01** (maestro):

 Bash

```
sudo mariadb lab_bdd << 'EOF'
INSERT INTO clientes (nombre, apellido, email, telefono, region, ciudad)
VALUES ('Test', 'Replicacion', 'test.repl@lab.test', '5550000001',
'norte', 'CDMX');
-- Confirmar que la fila existe en el maestro
SELECT id, nombre, apellido, region, ciudad
FROM   clientes
WHERE  email = 'test.repl@lab.test';
EOF
```

En `bdd-nodo02` (esclavo), verificar que la fila apareció sin haberla insertado ahí:

 Bash

```
sudo mariadb lab_bdd -e "
  SELECT id, nombre, apellido, region, ciudad
  FROM   clientes
  WHERE  email = 'test.repl@lab.test';
"
```

Prueba 2 — UPDATE

En `bdd-nodo01`:

 Bash

```
sudo mariadb lab_bdd -e "
  UPDATE clientes
  SET     ciudad = 'Guadalajara', region = 'oeste'
  WHERE  email  = 'test.repl@lab.test';
"
```

En `bdd-nodo02`, verificar el cambio:

 Bash

```
sudo mariadb lab_bdd -e "  
    SELECT id, nombre, ciudad, region  
    FROM    clientes  
    WHERE   email = 'test.repl@lab.test';  
"
```

La ciudad debe ser `Guadalajara` y la región `oeste` .

Prueba 3 — DELETE

En `bdd-nodo01` :

 Bash

```
sudo mariadb lab_bdd -e "  
    DELETE FROM clientes WHERE email = 'test.repl@lab.test';  
"
```

En `bdd-nodo02` , verificar que la fila desapareció:

 Bash

```
sudo mariadb lab_bdd -e "  
    SELECT COUNT(*) AS debe_ser_cero  
    FROM    clientes  
    WHERE   email = 'test.repl@lab.test';  
"
```

Prueba 4 — DDL (ALTER TABLE)

En `bdd-nodo01` , agregar una columna de auditoría:

 Bash

```
sudo mariadb lab_bdd -e "  
  ALTER TABLE clientes  
    ADD COLUMN ultima_modificacion DATETIME  
      DEFAULT CURRENT_TIMESTAMP  
      ON UPDATE CURRENT_TIMESTAMP  
      AFTER fecha_alta;  
"
```

En `bdd-nodo02`, verificar que la columna se replicó:

 Bash

```
sudo mariadb lab_bdd -e "SHOW COLUMNS FROM clientes  
LIKE 'ultima_modificacion';"
```

Prueba 5 — Verificación de conteo global de datos

Ejecutar en **ambos nodos** y comparar los resultados; deben ser idénticos:

 Bash

```
sudo mariadb -e "  
SELECT 'clientes'          AS tabla, COUNT(*) AS filas FROM lab_bdd.clientes  
UNION ALL  
SELECT 'productos',        COUNT(*)          FROM  
lab_bdd.productos          UNION ALL  
SELECT 'pedidos',          COUNT(*)          FROM lab_bdd.pedidos  
UNION ALL  
SELECT 'detalle_pedidos',  COUNT(*)          FROM lab_bdd.detalle_pedidos;  
"
```

Salida esperada (idéntica en nodo01 y nodo02):

Text

```
+-----+-----+
| tabla          | filas |
+-----+-----+
| clientes       | 20    |
| productos      | 10    |
| pedidos        | 20    |
| detalle_pedidos | 36    |
+-----+-----+
```

D.12 Prueba de `read_only` en el esclavo (VM — nodo02)

Bash

```
# Intento con app_user (no tiene SUPER)
mariadb -u app_user -p'AppUser_2025!' lab_bdd \
-e "INSERT INTO clientes (nombre, apellido, email, region)
VALUES ('Intento', 'Directo', 'intento@lab.test', 'norte');" \
2>&1
# Intento con lab_admin (tampoco tiene SUPER)
mariadb -u lab_admin -p'LabAdmin_2025!' lab_bdd \
-e "INSERT INTO clientes (nombre, apellido, email, region)
VALUES ('Intento', 'Admin', 'intento.admin@lab.test', 'norte');" \
2>&1
```

Ambos intentos deben producir el error:

Text

```
ERROR 1290 (HY000): The MariaDB server is running with the --read-
only option
so it cannot execute this statement
```

D.13 Monitoreo de lag y posición GTID (VM — ambos nodos)

 Bash

```
# En nodo01: posición GTID actual del binlog del maestro
sudo mariadb -e "SELECT @@global.gtid_binlog_pos
AS posicion_gtid_maestro;"
sudo mariadb -e "SHOW MASTER STATUS\G"
# En nodo02: posición GTID aplicada por el esclavo y estado de hilos
sudo mariadb -e "SELECT @@global.gtid_slave_pos AS posicion_gtid_esclavo;"
sudo mariadb -e "SHOW SLAVE STATUS\G" | \
    grep -E
'Slave_IO_Running|Slave_SQL_Running|Seconds_Behind|Gtid|Last.*Error'
```

Para monitoreo continuo del lag (se actualiza cada 2 segundos; **Ctrl+C** para salir):

 Bash

```
watch -n 2 "sudo mariadb -e 'SHOW SLAVE STATUS\G' 2>/dev/null | \
    grep -E
'Slave_IO_Running|Slave_SQL_Running|Seconds_Behind_Master|Gtid_IO_Pos'"
```

El `gtid_slave_pos` del esclavo debe coincidir con el `gtid_binlog_pos` del maestro cuando `Seconds_Behind_Master` es `0`, indicando que no hay retraso.

D.14 Apagar ambas VMs y tomar el snapshot `fase10-completa` (host)

Desde las sesiones SSH de cada nodo:

 Bash

```
# En la terminal SSH de nodo01
sudo poweroff
# En la terminal SSH de nodo02
sudo poweroff
```

Confirmar desde el host que ambas VMs están detenidas:

 PowerShell

```
VBoxManage list runningvms
```

La salida debe estar vacía. Tomar los snapshots:

 PowerShell

```
VBoxManage snapshot "bdd-nodo01" take "fase10-completa" `
  --description "MAESTRO: binlog ROW + GTID habilitado. server_id=1. repl_user
creado para 192.168.56.102. bind-address=0.0.0.0. Pruebas
INSERT/UPDATE/DELETE/DDL verificadas."
VBoxManage snapshot "bdd-nodo02" take "fase10-completa" `
  --description "ESCLAVO: read_only=ON. server_id=2. CHANGE MASTER TO con
MASTER_USE_GTID=slave_pos hacia 192.168.56.101. Slave_IO=Yes, Slave_SQL=Yes.
Columna ultima_modificacion replicada vía DDL."
```

Confirmar los snapshots:

 PowerShell

```
VBoxManage snapshot "bdd-nodo01" list
VBoxManage snapshot "bdd-nodo02" list
```

Cada VM debe mostrar seis snapshots: `fase05-completa` hasta `fase10-completa`.

E. Verificación de funcionamiento

Esta fase se considera completa cuando se cumplen **todos** los puntos siguientes en el estado de los nodos encendidos:

1. `SHOW VARIABLES LIKE 'log_bin'` devuelve `ON` en `bdd-nodo01`.
2. `SHOW BINARY LOGS` muestra al menos `mariadb-bin.000001` en `bdd-nodo01`.
3. `SHOW VARIABLES LIKE 'binlog_format'` devuelve `ROW` en `bdd-nodo01`.
4. `SHOW VARIABLES LIKE 'server_id'` devuelve `1` en `bdd-nodo01` y `2` en `bdd-nodo02`.
5. `SHOW GRANTS FOR 'repl_user'@'192.168.56.102'` en `bdd-nodo01` muestra exactamente `GRANT REPLICATION SLAVE ON *.* TO 'repl_user'@'192.168.56.102'`.
6. `SHOW SLAVE STATUS\G` en `bdd-nodo02` muestra:
 - `Slave_IO_Running: Yes`

- `Slave_SQL_Running: Yes`
- `Seconds_Behind_Master: 0`
- `Last_IO_Error:` (vacío)
- `Last_SQL_Error:` (vacío)
- `Using_Gtid: Slave_Pos`

7. `SHOW VARIABLES LIKE 'read_only'` devuelve `ON` en `bdd-nodo02`.
 8. La columna `ultima_modificacion` (añadida en la Prueba 4 de DDL) existe en la tabla `clientes` de **ambos** nodos.
 9. Los conteos de las cuatro tablas de `lab_bdd` son idénticos en ambos nodos (20 / 10 / 20 / 36 registros).
 10. Los intentos de INSERT directo con `app_user` y `lab_admin` en `bdd-nodo02` fallan con `ERROR 1290: read-only`.
 11. El `gtid_slave_pos` del esclavo coincide con el `gtid_binlog_pos` del maestro.
 12. Los snapshots `fase10-completa` existen en `bdd-nodo01` y `bdd-nodo02`.
 13. El estudiante puede explicar sin ver el documento la diferencia entre el hilo I/O y el hilo SQL, y qué ocurre si solo uno de los dos falla.
-

F. Problemas comunes y soluciones

Problema	Causa probable	Solución
<code>Slave_IO_Running: No ,</code> <code>Last_IO_Error: "Can't connect to MySQL server on '192.168.56.101'"</code>	<code>bind-address</code> sigue siendo <code>127.0.0.1</code> en nodo01; el <code>sed</code> no se aplicó o el servicio no se reinició	En nodo01: <code>grep bind-address /etc/mysql/mariadb.conf.d/50-server.cnf</code> ; si aún muestra <code>127.0.0.1</code> , ejecutar el <code>sed</code> de nuevo y <code>sudo systemctl restart mariadb</code>
<code>Slave_IO_Running: No ,</code> <code>Last_IO_Error: "Access denied for user 'repl_user'@'192.168.56.102'"</code>	El usuario fue creado con host incorrecto, o la contraseña difiere	En nodo01: <code>SELECT User, Host FROM mysql.user WHERE User='repl_user';</code> ; si el host es incorrecto, <code>DROP USER 'repl_user'@'host_incorrecto';</code> y recrear con la IP exacta
<code>Slave_SQL_Running: No ,</code> <code>Last_SQL_Error: "Duplicate entry 'X' for key 'PRIMARY'"</code>	Se ejecutó DML en nodo01 después de habilitar el binlog y antes de que el esclavo iniciara; el esclavo intenta insertar datos que ya existen	Para omitir ese evento puntual (solo si no representa pérdida de datos crítica): <code>STOP SLAVE;</code> <code>SET GLOBAL SQL_SLAVE_SKIP_COUNTER = 1;</code> <code>START SLAVE;</code> Si el problema se repite, ver el reto avanzado para resincronización completa
<code>Slave_IO_Running: No ,</code> <code>Last_IO_Error: "error connecting ... errno: 111 Connection refused"</code>	Puerto 3306 bloqueado por firewall (ufw o iptables activo en nodo01)	En nodo01: <code>sudo ufw status</code> ; si está activo: <code>sudo ufw allow from 192.168.56.102 to any port 3306 proto tcp</code>
MariaDB no arranca en nodo01 después de crear <code>60-replication-master.cnf</code>	Error de sintaxis en el archivo de configuración	<code>sudo journalctl -u mariadb --no-pager -n 30</code> ; buscar línea <code>[ERROR]</code> o <code>unknown variable</code> ; corregir el parámetro indicado
<code>SHOW SLAVE STATUS</code> muestra <code>Using_Gtid: No</code> en lugar de <code>Slave_Pos</code>	El <code>CHANGE MASTER TO</code> se ejecutó sin <code>MASTER_USE_GTID = slave_pos</code>	<code>STOP SLAVE; CHANGE MASTER TO MASTER_USE_GTID = slave_pos; START SLAVE;</code>
El <code>sed</code> no encontró la línea <code>bind-address</code> en <code>50-server.cnf</code>	El parámetro está comentado o en otro archivo	<code>grep -r bind-address /etc/mysql/</code> para localizarlo; descomentarlo o agregar <code>bind-address = 0.0.0.0</code> explícitamente en <code>60-replication-master.cnf</code>

<code>Seconds_Behind_Master</code> crece indefinidamente sin llegar a 0	El hilo SQL no puede aplicar eventos tan rápido como el I/O los descarga	Verificar con <code>SHOW PROCESSLIST</code> en nodo02 qué está haciendo el hilo SQL (<code>System lock</code> , <code>Updating</code> , etc.); comprobar que el disco de nodo02 no está saturado con <code>iostat -x 1</code>
<code>SHOW GRANTS FOR 'repl_user'@'192.168.56.102'</code> también aparece en nodo02	Los eventos <code>CREATE USER</code> y <code>GRANT</code> del maestro se replicaron al esclavo, como es esperado	Comportamiento correcto. El usuario <code>repl_user</code> en nodo02 es inofensivo; no tiene privilegios dañinos y no puede ser usado para configurar una replicación entrante al esclavo
<code>read_only</code> no bloquea escrituras del usuario <code>root</code>	<code>root</code> tiene el privilegio <code>SUPER</code> , que está exento de <code>read_only</code> por diseño	Comportamiento correcto: <code>read_only</code> protege de escrituras accidentales de aplicaciones, no de administradores. Para bloquear también a <code>SUPER</code> , usar <code>SET GLOBAL super_read_only = ON</code> (con precaución, bloquea al hilo SQL si se activa antes de que esté conectado)
El <code>ALTER TABLE</code> de la Prueba 4 falla en nodo02 con "Column already exists"	La columna <code>ultima_modificacion</code> se añadió directamente en nodo02 antes de esta prueba	En nodo02: <code>STOP SLAVE; SET GLOBAL SQL_SLAVE_SKIP_COUNTER = 1; START SLAVE;</code> . En el futuro, nunca ejecutar DDL directamente en el esclavo
MariaDB en nodo02 arranca pero la replicación no se inicia automáticamente tras un reinicio	<code>START SLAVE</code> no persiste entre reinicios si hay un error de hilo al arrancar	Verificar <code>SHOW SLAVE STATUS\G</code> después del reinicio; ejecutar <code>START SLAVE</code> manualmente si los hilos están en <code>No</code> . Para arranque automático: <code>--skip-slave-start=0</code> en la configuración (por defecto los hilos arrancan automáticamente en MariaDB 10.11)

G. Checklist de validación

- ☐ `grep bind-address /etc/mysql/mariadb.conf.d/50-server.cnf` devuelve `0.0.0.0` en `bdd-nodo01`.

- ☐ El archivo `60-replication-master.cnf` existe en `bdd-nodo01` con `server_id = 1`, `log_bin`, `binlog_format = ROW` y `gtid_domain_id = 1`.
 - ☐ `SHOW VARIABLES LIKE 'log_bin'` devuelve `ON` en `bdd-nodo01`.
 - ☐ `SHOW BINARY LOGS` muestra al menos el archivo `mariadb-bin.000001` en `bdd-nodo01`.
 - ☐ `SHOW GRANTS FOR 'repl_user'@'192.168.56.102'` muestra `GRANT REPLICATION SLAVE ON *.*` en `bdd-nodo01` y el usuario no existe con host distinto de `192.168.56.102`.
 - ☐ El archivo `60-replication-slave.cnf` existe en `bdd-nodo02` con `server_id = 2`, `relay_log` y `read_only = ON`.
 - ☐ `SHOW VARIABLES LIKE 'server_id'` devuelve `1` en `nodo01` y `2` en `nodo02`.
 - ☐ `SHOW VARIABLES LIKE 'read_only'` devuelve `ON` en `bdd-nodo02`.
 - ☐ `SHOW SLAVE STATUS\G` en `bdd-nodo02` muestra `Slave_IO_Running: Yes`.
 - ☐ `SHOW SLAVE STATUS\G` en `bdd-nodo02` muestra `Slave_SQL_Running: Yes`.
 - ☐ `Seconds_Behind_Master` es `0` en `bdd-nodo02`.
 - ☐ `Last_IO_Error` y `Last_SQL_Error` están vacíos en `bdd-nodo02`.
 - ☐ `Using_Gtid: Slave_Pos` aparece en `SHOW SLAVE STATUS` de `bdd-nodo02`.
 - ☐ El INSERT de prueba (`test.repl@lab.test`) apareció en `bdd-nodo02` sin ejecutarlo ahí directamente.
 - ☐ El UPDATE (ciudad → `Guadalajara`) se reflejó en `bdd-nodo02`.
 - ☐ El DELETE eliminó la fila en `bdd-nodo02` (conteo = 0).
 - ☐ La columna `ultima_modificacion` existe en `clientes` de **ambos** nodos (propagada por el ALTER TABLE en el maestro).
 - ☐ Los conteos de las cuatro tablas de `lab_bdd` son idénticos en `nodo01` y `nodo02`.
 - ☐ El intento de INSERT con `app_user` en `bdd-nodo02` falla con error `read-only`.
 - ☐ El intento de INSERT con `lab_admin` en `bdd-nodo02` falla con error `read-only`.
 - ☐ Los snapshots `fase10-completa` existen en `bdd-nodo01` y `bdd-nodo02` y cada VM muestra seis snapshots con `VBoxManage snapshot list`.
 - ☐ Puedo explicar sin ver el documento la diferencia entre el hilo I/O y el hilo SQL del esclavo, y qué ocurre si solo uno de los dos falla.
 - ☐ Puedo explicar qué es `Seconds_Behind_Master` y qué implica un valor distinto de cero.
 - ☐ Puedo explicar por qué `read_only` no afecta al hilo SQL de replicación pero sí a `app_user`.
-

Preguntas teóricas para estudiantes

1. El parámetro `log_slave_updates = ON` se habilitó tanto en el maestro como en el esclavo, aunque en esta fase el esclavo no tiene ningún esclavo propio. Explica en qué escenario futuro —dentro de las fases restantes del laboratorio o en una arquitectura real— sería indispensable que el esclavo también tenga su propio binary log activo. ¿Qué operación de administración (failover, clonación, auditoría) se volvería imposible o muy difícil sin él?
2. La replicación de MariaDB en este laboratorio es **asíncrona**. Describe un escenario concreto de fallo —no abstracto, sino usando los nodos de este laboratorio— en el que el asincronismo resulta en pérdida de datos: ¿qué operación debe haberse ejecutado en `bdd-nodo01`, cuándo debe fallar el nodo, y qué dato concreto de `lab_bdd` se perdería en `bdd-nodo02`? ¿Qué mecanismo de MariaDB (sin hardware redundante extra) mitigaría parcialmente este riesgo?
3. Compara la replicación basada en **posición de archivo** (`MASTER_LOG_FILE` + `MASTER_LOG_POS`) con la basada en **GTID** (`MASTER_USE_GTID = slave_pos`). Si el maestro rota el binary log (crea `mariadb-bin.000002`) exactamente cuando el esclavo está leyendo el último evento de `mariadb-bin.000001`, ¿cómo maneja cada modalidad esa transición? ¿Qué ventaja operacional concreta ofrece el GTID si en el futuro hay que re apuntar el esclavo a un maestro de sustitución?
4. En el esclavo, `read_only = ON` impide escrituras de usuarios normales, pero el hilo SQL de replicación puede seguir aplicando cambios. Explica bajo qué usuario interno de MariaDB opera el hilo SQL y qué privilegios tiene que lo eximen de `read_only`. ¿Qué ocurriría con la replicación si se activara `super_read_only = ON` en el esclavo *antes* de que el hilo SQL se haya iniciado? ¿Y si se activa *después* de que el hilo SQL ya está corriendo?
5. El formato ROW registra las filas afectadas (imágenes antes y después del cambio) en lugar de la sentencia SQL original. Analiza las implicaciones para la sentencia `UPDATE pedidos SET total = total * 1.10 WHERE region = 'norte'` si hay 10 pedidos en la región norte: ¿cuántos eventos ROW generaría esta sentencia en el binlog? ¿Cómo se compararía el tamaño del evento en formato ROW versus STATEMENT? ¿En qué circunstancia podría el formato STATEMENT producir un resultado distinto en el esclavo al aplicar esta misma sentencia?

Ejercicios prácticos

1. **Observación del binary log con `mysqlbinlog`.**
En `bdd-nodo01`, ejecutar un INSERT, un UPDATE y un DELETE sobre `lab_bdd`. Luego usar `mysqlbinlog` para leer el archivo de binary log y localizar los eventos:

 Bash

```
sudo mysqlbinlog --base64-output=DECODE-ROWS --verbose \  
/var/log/mysql/mariadb-bin.000001 | \  
grep -A 20 "### INSERT\|### UPDATE\|### DELETE"
```

Responder: ¿se ve la sentencia SQL original o los valores de columnas? ¿Puedes identificar el `server_id`, el timestamp y el GTID de cada transacción? ¿Cómo aparece el ALTER TABLE de la Prueba 4 en el log (como evento ROW o de otro tipo)? Documentar los hallazgos con la salida del comando.

2. Simulación de fallo del hilo SQL y recuperación.

En `bdd-nodo02`, detener únicamente el hilo SQL manteniendo el I/O activo:

 Bash

```
sudo mariadb -e "STOP SLAVE SQL_THREAD;"
```

En `bdd-nodo01`, ejecutar 5 INSERT en `lab_bdd.clientes`. Verificar que `Seconds_Behind_Master` crece en `bdd-nodo02` (el I/O sigue descargando pero el SQL no aplica). Reiniciar el hilo SQL y observar cómo el lag cae a 0:

 Bash

```
sudo mariadb -e "START SLAVE SQL_THREAD;"  
watch -n 1 "sudo mariadb -e 'SHOW SLAVE STATUS\G' |  
grep Seconds_Behind_Master"
```

Documentar los valores de `Seconds_Behind_Master` a lo largo del ejercicio y explicar el comportamiento observado. Al finalizar, eliminar las 5 filas de prueba con `DELETE FROM lab_bdd.clientes WHERE id > 20`.

3. Script de verificación de consistencia maestro-esclavo.

Escribir un script Bash en `bdd-nodo01` que: (a) inserte 30 filas de prueba en `lab_bdd.clientes` con valores de `region` aleatorios distribuidos entre los cuatro valores del ENUM; (b) espere 3 segundos; © compare automáticamente el conteo de filas en nodo01 (`mysql --host=127.0.0.1`) y en nodo02 (`mysql --host=192.168.56.102 -u repl_user -p'ReplUser_2025!'`) — para esta prueba, otorgar temporalmente el privilegio `SELECT` a `repl_user` en `lab_bdd.clientes`; (d) imprima `CONSISTENTE: maestro=X, esclavo=X` si los conteos coinciden o `DIVERGENCIA: maestro=X, esclavo=Y` si difieren. Ejecutar el script y documentar el resultado. Al finalizar, eliminar las filas de prueba y revocar el privilegio temporal de `repl_user`.

Reto adicional para alumnos avanzados

Implementar el procedimiento de **resincronización completa del esclavo desde cero**, aplicable cuando el esclavo se desincronizó gravemente (por ejemplo, alguien ejecutó `DROP TABLE pedidos` directamente en `bdd-nodo02` o modificó filas manualmente). El procedimiento debe poder ejecutarse sin detener el maestro ni interrumpir a los clientes que escriben en `bdd-nodo01`. Pasos a documentar con comandos exactos:

1. Detener los hilos de replicación en el esclavo (`STOP SLAVE`).
2. Generar un volcado consistente del maestro con `mysqldump` , incluyendo la posición GTID del punto en que se tomó el volcado (investigar las opciones específicas de `mysqldump` de MariaDB 10.11 que registran la información GTID en el encabezado del archivo; comparar `--master-data=2` , `--dump-slave` y la variable de sesión `@@global.gtid_binlog_pos` anotada manualmente antes del volcado).
3. Restaurar el volcado en el esclavo.
4. Reconfigurar `CHANGE MASTER TO` con la posición GTID exacta registrada en el paso 2.
5. Iniciar los hilos de replicación y verificar con `SHOW SLAVE STATUS\G` que ambos hilos muestran `Yes` , el lag es `0` y `Last_SQL_Error` está vacío.

Incluir en la entrega: los comandos exactos con su salida, el razonamiento técnico de cada decisión (en particular, por qué se usa `--single-transaction` y qué garantía ofrece), y la diferencia entre resincronizar desde un volcado `mysqldump` versus desde un snapshot de VirtualBox del nodo01 en `fase08-completa` .

Criterios de evaluación para el profesor

Criterio	Peso	Indicador de logro
Configuración del maestro	20%	<code>SHOW VARIABLES</code> confirma <code>log_bin=ON</code> , <code>binlog_format=ROW</code> , <code>server_id=1</code> , <code>gtid_domain_id=1</code> ; <code>60-replication-master.cnf</code> existe con el contenido correcto; <code>repl_user</code> tiene exactamente <code>REPLICATION SLAVE ON *.*</code> para la IP correcta del esclavo
Configuración del esclavo	20%	<code>SHOW VARIABLES</code> confirma <code>server_id=2</code> , <code>read_only=ON</code> ; <code>60-replication-slave.cnf</code> existe; <code>SHOW SLAVE STATUS</code> muestra <code>Slave_IO_Running: Yes</code> , <code>Slave_SQL_Running: Yes</code> , <code>Using_Gtid: Slave_Pos</code>
Pruebas de propagación	25%	El estudiante ejecuta y documenta las cinco pruebas (INSERT, UPDATE, DELETE, DDL, conteo global) y muestra evidencia de que cada cambio en el maestro se reflejó en el esclavo; puede presentar los resultados de ambos nodos simultáneamente
Verificación de <code>read_only</code>	10%	Se demuestra con evidencia del mensaje de error que usuarios sin SUPER no pueden escribir en el esclavo; el estudiante explica por qué el hilo SQL es la excepción y el usuario <code>root</code> también lo es
Comprensión conceptual (preguntas teóricas)	20%	Las respuestas demuestran comprensión de los hilos de replicación, GTID vs. file+pos, asincronismo y <code>read_only</code> usando terminología técnica precisa con ejemplos del propio laboratorio
Snapshots y documentación	5%	Los snapshots <code>fase10-completa</code> existen en ambos nodos con descripciones informativas; el estudiante puede listar los seis snapshots de cada VM con <code>VBoxManage snapshot list</code>

Preparación para la siguiente fase

La Fase 11: Replicación Lógica (Multi-Maestro con `bdd-nodo03`) requerirá:

- La replicación física entre `bdd-nodo01` y `bdd-nodo02` funcionando correctamente (esta fase), con snapshots `fase10-completa` tomados en ambos nodos.
- Comprensión sólida de GTID y binary log: la Fase 11 reutilizará exactamente los mismos conceptos pero en una topología bidireccional donde nodo03 actúa como maestro simultáneo de nodo01.
- **Creación de `bdd-nodo03`** : será la primera VM nueva del laboratorio desde la Fase 5. Se creará como un **clon enlazado** (*linked clone*) de `bdd-nodo01` tomado del snapshot `fase08-completa` —que tiene `lab_bdd` instalado pero sin configuración de replicación— y se le asignará la IP `192.168.56.103` y el hostname `bdd-nodo03` . El procedimiento de clonación y configuración inicial del nuevo nodo se hará íntegramente al inicio de la Fase 11.
- El Documento de Diseño Distribuido (`fase09-disenyo-distribuido.md` , sección 2) como referencia: define que nodo03 tendrá el esquema `lab_bdd` completo y replicación bidireccional con nodo01.

No es necesario instalar, configurar ni crear nada adicional ahora.
